

下面给你一版正式可执行、适合项目归档的《LoveAdvisor V2.5 正式实施方案》。这版不是延续之前那种“按节点继续补”，而是基于你们已经暴露出来的真实问题，转向评测驱动的**稳定化开发**。其核心目的，是解决当前开发低效、测试手工化、规则优化缺乏量化依据的问题。方案是在 V2 已完成架构重整、信号层拆分、R1 规则化、S5 重接入、Day 7 全量评估基础上提出的。

LoveAdvisor V2.5 正式实施方案

评测驱动的稳定化开发阶段

一、项目阶段重新定义

1.1 当前阶段判断

LoveAdvisor 项目截至 V2 Day 7，已经完成了以下关键工作：

- 完成 V1 的链路打通验证
- •
完成 V2 的架构重整与职责重构
- •
建立 S2 基础互动信号层
- •
建立 S3 高级关系信号层
- •

实现 R1 后端规则引擎

-
-

完成 S5 建议层与规则层对齐

-
-

打通 O1 输出层与前后端展示链路

-
-

初步形成全量测试与错误分类体系

-

因此，项目已经不再处于“从 0 到 1 搭系统”的阶段，而是进入了一个新的阶段：

系统已可运行，但尚未形成高效、科学、可持续的优化机制。

当前的主要瓶颈不再是“链路通不通”，而是：

-

批量测试方式低效，严重依赖手工

-
-

规则优化缺少自动化评测支撑

-
-

E 类边界、interest_level、next_step 仍存在系统性误差

-
-

前端承担了不该承担的测试工作

-
-

迭代方式仍偏经验驱动，而非指标驱动

•

基于此，项目正式进入：

LoveAdvisor V2.5：评测驱动的稳定化开发阶段

1.2 V2.5 的核心目标

V2.5 的目标不是继续扩功能，也不是继续堆节点，而是解决一个更根本的问题：

让项目从“能迭代”升级为“能高效、可量化、可持续地迭代”。

具体包括四个目标：

目标编号	目标名称	核心内容
G1	测试工程化	将 Day 7 式手工评测改为自动评测
G2	规则稳定化	围绕核心误差簇修复规则边界
G3	建议收敛化	压缩建议层尤其是 next_step 的默认推进倾向
G4	产品轻收口	在稳定化之后再优化展示与用户体验

1.3 V2.5 与 V1 / V2 的区别

阶段	目标	主导方式	核心产物
V1	打通链路、验证可行性	workflow 驱动	可运行 Demo
V2	重构职责、规则增强	架构驱动	稳定主链路
V2.5	量化优化、提升迭代效率	评测驱动	自动评测体系 + 稳定化版本

一句话概括：

V1 解决“能不能做”，V2 解决“怎么做稳”，V2.5 解决“怎么高效地把它做好”。

二、V2.5 的总体开发原则

2.1 原则一：开发与评测彻底分离

当前开发中最明显的低效点，是把“前端手工输入样本”当成评测方法。这会导致：

-

批量测试效率极低

-

-

结果整理重复劳动严重

-

-

每轮改动无法快速回归

-

-

误差分析高度依赖人工体力

-

因此，V2.5 的第一原则是：

前端负责展示，脚本负责评测。

以后开发流程必须拆为三层：

-

开发层：改 prompt、改规则、改后端逻辑

-

-

评测层：自动跑样本、自动生成表格、自动分类错误

-

-

展示层：仅查看少量代表样本的产品表现

-

2.2 原则二：优化必须由指标驱动

Day 7 已经形成了比较清晰的判定维度：

-

R1 是否正确

-
-

建议是否对齐

-
-

next_step 是否克制

-
-

问题类型分类

-

V2.5 要把它们从“人工复盘口径”升级为“正式开发指标”。

建议固定以下四项核心指标：

指标	含义	作用
Stage 准确率	relationship_stage 是否正确	判断主标签稳定性
R1 严格准确率	stage + interest_level 同时正确	判断规则层整体质量
建议对齐率	S5 是否服从 R1 结果	判断建议层是否越权
next_step 克制率	最终动作是否符合阶段边界	判断产品表达是否成熟

所有后续优化，必须明确对应至少一个指标提升目标。

2.3 原则三：按误差簇优化，而不是按节点机械推进

V2 阶段是按节点推进的，这在架构建设期是合理的。
但到了 V2.5，继续机械地“今天改 S2、明天改 S3”会越来越低效。

因为 Day 7 已经证明，当前问题已经收敛到几个明确误差簇：

- E 类边界失守
- •
interest_level 中间档不稳
- •
next_step 普遍不够克制
-

因此，V2.5 的优化顺序必须是：

先看误差集中在哪，再决定修哪一层。

2.4 原则四：不新增复杂链路，优先固化现有链路

当前主链路已经基本形成：

前端输入 → 后端调用 → Coze 工作流 → S2/S3 → R1 → S5 → O1 → 前端展示。

V2.5 阶段不再优先新增新节点、新模型分工、新产品模块。
所有工作优先围绕以下三件事展开：

- 测试是否能自动跑
-
-

规则是否能稳定修

-
-

表达是否能明显收敛

-

三、V2.5 总体架构设计

3.1 业务主链路保持不变

V2.5 不重构业务主链路，继续沿用 V2 已验证的结构：

用户输入

- S1 文本清洗
- S2 基础互动信号提取
- S3 高级关系信号提取
- R1 后端规则引擎
- S5 策略建议生成
- O1 输出组装层
- 前端展示

原因是：

当前主问题并不在“链路设计错误”，而在“链路优化与评测方式低效”。

3.2 新增“评测子系统”

V2.5 最大的新增部分，不是业务链路，而是旁路的评测系统。

新增结构如下：

测试集文件

- 批量评测脚本
- 自动请求 /analyze
- 自动提取返回结果
- 自动生成测试结果表
- 自动统计核心指标

→ 自动输出错误分类

这套子系统不影响产品主链路，但直接决定开发效率。

3.3 评测系统的定位

评测系统的职责包括：

- 管理标准测试样本
- •
固定预期结果
- •
一键跑全量样本
- •
自动生成结果记录
- •
自动判定主要指标
- •
为规则修改提供客观对比依据
-

它的作用不是“替代人工判断”，而是：

把机械重复劳动从人身上剥离出来，让人只做真正需要判断的事情。

四、V2.5 核心模块设计

4.1 T1：测试集管理模块

职责

统一管理所有回归测试样本及其预期结果。

输入结构建议

每条样本至少包含：

```
{
  "sample_id": "A1",
  "group": "A",
  "input_text": "A: 你好\nB: 你好\nA: 你是哪个专业的\nB: 金融",
  "expected_relationship_stage": "初识期",
  "expected_interest_level": "低",
  "expected_suggestion_aligned": true,
  "expected_next_step_style": "克制"
}
```

说明

V2 阶段已有 A/B/C/D/E 五组测试资产，Day 7 也已经形成了 15 条样本的完整测试表。V2.5 的任务不是重新设计样本，而是把现有测试资产正式结构化沉淀下来。

验收标准

-

A1-E3 样本全部入库

-

-

字段统一

-

-

支持脚本直接读取

-

-

后续可扩展新增样本

-

4.2 T2：批量评测执行模块

职责

读取测试集，自动请求后端接口，采集结果并落表。

执行流程

读取测试集
→ 遍历样本
→ POST /analyze
→ 获取最终 JSON
→ 提取核心字段
→ 写入结果文件

输出结果建议字段

字段	含义
sample_id	样本编号
expected_stage	预期阶段
actual_stage	实际阶段
expected_interest	预期兴趣等级
actual_interest	实际兴趣等级
r1_correct	R1 是否严格正确
suggestion_aligned	建议是否对齐
next_step_conservative	next_step 是否克制
issue_type	问题类型
raw_output	原始返回结果

验收标准

-

15 条样本可一键批量跑完

-

-

无需人工逐条输入前端

-

-

自动产出表格文件

-

-

输出结构稳定

-

4.3 T3：指标统计与错误分类模块

职责

对批量测试结果进行自动统计与基础归类。

核心指标

-

Stage 准确率

-

-

R1 严格准确率

-

-

建议对齐率

-
-

next_step 克制率

-

当前基线

Day 7 当前结果为：

-

R1 严格准确率：46.7%

-
-

建议对齐率：93.3%

-
-

next_step 克制率：40.0%

-
-

全链路通过率：40.0%

-

这些数值应作为 V2.5 第一轮优化的对照基线。

错误分类建议

至少先支持以下类型：

-

边界失守

-
-

误判初识

-
-

interest_level 低估

-
-

interest_level 高估

-
-

建议漂移

-
-

next_step 不克制

-

验收标准

-

能自动计算核心指标

-
-

能自动初步打标签

-
-

能支持人工二次复核

-

4.4 R1：规则稳定化优化模块

职责

基于自动评测结果，围绕误差簇修复规则边界。

当前优先问题

根据 Day 7，总体优先级如下：

优先级	问题	说明
P1	E 类边界失守	E2/E3 被误吸入初识期
P2	interest_level 分档不稳	中间档位容易上下漂移
P3	next_step 不克制	产品表达层问题最突出

优化要求

R1 不再做“大改”，而是做边界精修：

- 强化“无法判断”触发条件
- •
收紧“初识期兜底”的吸附范围
- •
重定义初识/暧昧/拉扯不同阶段的兴趣分档规则
-

验收标准

- E 类通过率显著提升
- •
R1 严格准确率高于 Day 7 基线
-

-
- 规则可解释、可回归、可复盘
-

4.5 S5: 建议收敛优化模块

职责

继续强化 S5 对 R1 的服从，并压缩 next_step 的默认推进倾向。

当前问题

Day 7 最突出的产品层问题，不是建议完全失控，而是：

next_step 过于容易默认“再推进一步”。

优化方向

建立更强的阶段约束：

阶段	next_step 要求
无法判断	观察，不加码
初识期	轻互动，不升级
暧昧期 / 中	顺势承接，不提速
拉扯期	降压，不追问
冷淡期	止损，先退一步

验收标准

-
- next_step 克制率明显提升
-
-
- 不再普遍出现默认推进
-
-

建议文风自然，但边界更稳

-

4.6 P1：前端展示轻收口模块

职责

在规则与建议相对稳定后，适配更自然的产品展示。

说明

前端不再承担批量测试角色，只承担：

-

页面展示验证

-

-

少量代表样本体验测试

-

-

输出阅读友好度优化

-

验收标准

-

字段展示清晰

-

-

状态区分明确

-

-

自然语言可读

-
-

不影响后端与评测系统独立运行

-

五、V2.5 分阶段实施安排

第 1 天：测试体系设计与测试集结构化

目标

完成 V2.5 的评测基础设施设计。

当天任务

1.

整理 A/B/C/D/E 样本

2.

3.

固定测试集字段协议

4.

5.

建立测试集 JSON/CSV 文件

6.

7.

明确自动评测输出字段

- 8.
- 9.

冻结核心指标统计口径

- 10.

当日产出

-

测试集文件

-
-

指标定义文档

-
-

错误分类字典初版

-

验收标准

-

15 条样本全部结构化

-
-

预期结果统一

-
-

不再依赖聊天记录人工找样本

-
-

第 2 天：批量评测脚本实现

目标

实现自动跑样本、自动落表。

当天任务

1.

编写脚本读取测试集

2.

3.

自动请求 `/analyze`

4.

5.

解析结果

6.

7.

输出 CSV/JSON 结果表

8.

9.

补充异常兜底与失败重试

10.

当日产出

-

batch_test.py

-

-

自动结果表

-
-

运行日志样例

-

验收标准

-

一键跑完整批测试

-
-

不用手工输前端

-
-

输出表格完整可用

-

第 3 天：指标统计与错误分类落地

目标

让每轮测试都能自动出结论，而不是靠人工复盘。

当天任务

1.

计算 Stage 准确率

2.

3.

计算 R1 严格准确率

4.

5.

计算建议对齐率

6.

7.

计算 next_step 克制率

8.

9.

自动生成初步错误分类

10.

当日产出

-

metrics_report.json / csv

-

-

错误分类统计表

-

-

当前版本基线报告

-

验收标准

-

跑完样本后自动出统计结果

-
-

与 Day 7 手工结果口径一致

-

第 4 天：E 类边界专项修复

目标

优先修复最危险的边界失守问题。

当天任务

1.

梳理 E1/E2/E3 差异

2.

3.

收紧“无法判断”触发条件

4.

5.

限制初识期兜底吸附

6.

7.

跑专项回归

8.

当日产出

-

R1 规则修订版

-
-

E 类专项测试表

-

验收标准

-

E 类通过率显著提升

-
-

E2/E3 不再轻易误吸入初识期

-

第 5 天：interest_level 分档专项修复

目标

解决当前 R1 最主要的精度问题。

当天任务

1.

复盘 A2/A3/B2/B3/C3 错误

2.

3.

重写中间档规则边界

4.

5.

增加“正向承接强度”与“后退信号”联动约束

6.

7.

批量回归验证

8.

当日产出

-

interest_level 新规则表

-

-

修复前后对比表

-

验收标准

-

R1 严格准确率明显提升

-

-

中间档位误差收敛

-

第 6 天：next_step 收敛专项修复

目标

解决当前最突出的产品表达问题。

当天任务

1.

复盘所有 next_step 不克制样本

2.

3.

调整 S5 Prompt 约束

4.

5.

增加阶段到动作的硬边界

6.

7.

重新批量测试

8.

当日产出

-

S5 Prompt 收敛版

-

-

next_step 专项对比表

-

验收标准

-

next_step 克制率明显提升

-
-

默认推进倾向下降

-

第 7 天：V2.5 阶段验收与产品轻收口

目标

完成 V2.5 第一轮阶段性验收。

当天任务

- 1.

重跑全量样本

- 2.
- 3.

对比 Day 7 基线

- 4.
- 5.

输出阶段总结报告

- 6.
- 7.

抽取少量样本检查前端展示

- 8.
- 9.

明确是否进入下一轮优化或产品展示阶段

- 10.

当日产出

-

V2.5 验收测试报告

-

-

核心指标前后对比

-

-

下一阶段建议

-

验收标准

-

自动评测链完整

-

-

R1 严格准确率高于基线

-

-

next_step 克制率高于基线

-

-

手工测试工作量显著下降

-

六、验收指标与阶段目标

6.1 V2.5 验收维度

维度	基线	V2.5 目标
自动评测覆盖率	0%	100%
R1 严格准确率	46.7%	明显高于基线
建议对齐率	93.3%	保持或更高
next_step 克制率	40.0%	明显高于基线
手工测试占比	很高	大幅下降

基线来自 Day 7 全量测试结果。

6.2 V2.5 的核心验收标准

若满足以下条件，则可认为 V2.5 阶段成功：

1.

批量评测可一键运行

2.

3.

测试结果可自动生成表格

4.

5.

指标可自动统计

6.

7.

E 类边界明显收紧

8.

9.

interest_level 分档明显改善

10.

11.

next_step 明显收敛

12.

13.

前端不再承担批量测试职责

14.

七、风险与控制策略

7.1 风险一：自动化评测落地后，人工仍不放心

控制策略：前两轮允许自动表 + 人工复核并行，逐步建立信任。

7.2 风险二：规则优化过度，导致其他样本退化

控制策略：每次修改后必须全量回归，而非只看专项样本。

7.3 风险三：S5 收敛过头，导致建议过于僵硬

控制策略：保持“边界更稳”优先于“语言更灵活”，但保留少量自然表达空间。

7.4 风险四：过早回到前端打磨，分散主矛盾

控制策略：前端展示优化只能排在自动评测和规则修复之后。

八、阶段结论

LoveAdvisor 当前最需要的，不是再写一版更长的 prompt，也不是继续靠前端逐条手工测样本，而是完成一次开发方法的升级。

V2 已经完成了主链路搭建与规则化重构。V2.5 的意义，在于把这条链路纳入一个可量化、可回归、可持续优化的开发体系中。其重点不是扩功能，而是补齐“测试工程化”这一缺口，并围绕 Day 7 已暴露的误差簇进行精准修复。

最终目标不是让项目“看起来更复杂”，而是让它：

- 更稳
- •
更准
- •
更省力
- •
更能持续迭代
-

九、最终阶段命名建议

建议将当前阶段正式命名为：

LoveAdvisor V2.5：评测驱动的稳定化开发阶段

这个命名比继续沿用“Day 8 / Day 9”更准确，因为它反映的不是节点开发顺序，而是项目开发方法论的升级。

如果你要，我下一条可以直接继续给你两样最落地的东西之一：

《V2.5 第一天实施清单》，或者 《自动评测系统的数据结构 + 脚本设计方案》。

下面给你一版正式可执行、适合项目归档的《LoveAdvisor V2.5 正式实施方案》。这版不是延续之前那种“按节点继续补”，而是基于你们已经暴露出来的真实问题，

转向评测驱动的稳定化开发。其核心目的，是解决当前开发低效、测试手工化、规则优化缺乏量化依据的问题。方案是在 V2 已完成架构重整、信号层拆分、R1 规则化、S5 重接入、Day 7 全量评估基础上提出的。

LoveAdvisor V2.5 正式实施方案

评测驱动的稳定化开发阶段

一、项目阶段重新定义

1.1 当前阶段判断

LoveAdvisor 项目截至 V2 Day 7，已经完成了以下关键工作：

- 完成 V1 的链路打通验证
- •
完成 V2 的架构重整与职责重构
- •
建立 S2 基础互动信号层
- •
建立 S3 高级关系信号层
- •
实现 R1 后端规则引擎

-
-

完成 S5 建议层与规则层对齐

-
-

打通 O1 输出层与前后端展示链路

-
-

初步形成全量测试与错误分类体系

-

因此，项目已经不再处于“从 0 到 1 搭系统”的阶段，而是进入了一个新的阶段：

系统已可运行，但尚未形成高效、科学、可持续的优化机制。

当前的主要瓶颈不再是“链路通不通”，而是：

-

批量测试方式低效，严重依赖手工

-
-

规则优化缺少自动化评测支撑

-
-

E 类边界、interest_level、next_step 仍存在系统性误差

-
-

前端承担了不该承担的测试工作

-
-

迭代方式仍偏经验驱动，而非指标驱动

•

基于此，项目正式进入：

LoveAdvisor V2.5：评测驱动的稳定化开发阶段

1.2 V2.5 的核心目标

V2.5 的目标不是继续扩功能，也不是继续堆节点，而是解决一个更根本的问题：

让项目从“能迭代”升级为“能高效、可量化、可持续地迭代”。

具体包括四个目标：

目标编号	目标名称	核心内容
G1	测试工程化	将 Day 7 式手工评测改为自动评测
G2	规则稳定化	围绕核心误差簇修复规则边界
G3	建议收敛化	压缩建议层尤其是 next_step 的默认推进倾向
G4	产品轻收口	在稳定化之后再优化展示与用户体验

1.3 V2.5 与 V1 / V2 的区别

阶段	目标	主导方式	核心产物
V1	打通链路、验证可行性	workflow 驱动	可运行 Demo
V2	重构职责、规则增强	架构驱动	稳定主链路
V2.5	量化优化、提升迭代效率	评测驱动	自动评测体系 + 稳定化版本

一句话概括：

V1 解决“能不能做”，V2 解决“怎么做稳”，V2.5 解决“怎么高效地把它做好”。

二、V2.5 的总体开发原则

2.1 原则一：开发与评测彻底分离

当前开发中最明显的低效点，是把“前端手工输入样本”当成评测方法。
这会导致：

- 批量测试效率极低
- •
结果整理重复劳动严重
- •
每轮改动无法快速回归
- •
误差分析高度依赖人工体力
-

因此，V2.5 的第一原则是：

前端负责展示，脚本负责评测。

以后开发流程必须拆为三层：

- **开发层：**改 prompt、改规则、改后端逻辑
 - •
 - 评测层：**自动跑样本、自动生成表格、自动分类错误
 - •
 - 展示层：**仅查看少量代表样本的产品表现
 -
-

2.2 原则二：优化必须由指标驱动

Day 7 已经形成了比较清晰的判定维度：

- R1 是否正确
- •
建议是否对齐
- •
next_step 是否克制
- •
问题类型分类
-

V2.5 要把它们从“人工复盘口径”升级为“正式开发指标”。

建议固定以下四项核心指标：

指标	含义	作用
Stage 准确率	relationship_stage 是否正确	判断主标签稳定性
R1 严格准确率	stage + interest_level 同时正确	判断规则层整体质量
建议对齐率	S5 是否服从 R1 结果	判断建议层是否越权
next_step 克制率	最终动作是否符合阶段边界	判断产品表达是否成熟

所有后续优化，必须明确对应至少一个指标提升目标。

2.3 原则三：按误差簇优化，而不是按节点机械推进

V2 阶段是按节点推进的，这在架构建设期是合理的。
但到了 V2.5，继续机械地“今天改 S2、明天改 S3”会越来越低效。

因为 Day 7 已经证明，当前问题已经收敛到几个明确误差簇：

- E 类边界失守
- •
interest_level 中间档不稳
- •
next_step 普遍不够克制
-

因此，V2.5 的优化顺序必须是：

先看误差集中在哪，再决定修哪一层。

2.4 原则四：不新增复杂链路，优先固化现有链路

当前主链路已经基本形成：

前端输入 → 后端调用 → Coze 工作流 → S2/S3 → R1 → S5 → O1 → 前端展示。

V2.5 阶段不再优先新增新节点、新模型分工、新产品模块。
所有工作优先围绕以下三件事展开：

- 测试是否能自动跑
- •
规则是否能稳定修
- •

表达是否能明显收敛

•

三、V2.5 总体架构设计

3.1 业务主链路保持不变

V2.5 不重构业务主链路，继续沿用 V2 已验证的结构：

用户输入

- S1 文本清洗
- S2 基础互动信号提取
- S3 高级关系信号提取
- R1 后端规则引擎
- S5 策略建议生成
- O1 输出组装层
- 前端展示

原因是：

当前主问题并不在“链路设计错误”，而在“链路优化与评测方式低效”。

3.2 新增“评测子系统”

V2.5 最大的新增部分，不是业务链路，而是旁路的评测系统。

新增结构如下：

测试集文件

- 批量评测脚本
- 自动请求 /analyze
- 自动提取返回结果
- 自动生成测试结果表
- 自动统计核心指标
- 自动输出错误分类

这套子系统不影响产品主链路，但直接决定开发效率。

3.3 评测系统的定位

评测系统的职责包括：

- 管理标准测试样本
- •
固定预期结果
- •
一键跑全量样本
- •
自动生成结果记录
- •
自动判定主要指标
- •
为规则修改提供客观对比依据
-

它的作用不是“替代人工判断”，而是：

把机械重复劳动从人身上剥离出来，让人只做真正需要判断的事情。

四、V2.5 核心模块设计

4.1 T1：测试集管理模块

职责

统一管理所有回归测试样本及其预期结果。

输入结构建议

每条样本至少包含：

```
{
  "sample_id": "A1",
  "group": "A",
  "input_text": "A: 你好\nB: 你好\nA: 你是哪个专业的\nB: 金融",
  "expected_relationship_stage": "初识期",
  "expected_interest_level": "低",
  "expected_suggestion_aligned": true,
  "expected_next_step_style": "克制"
}
```

说明

V2 阶段已有 A/B/C/D/E 五组测试资产，Day 7 也已经形成了 15 条样本的完整测试表。V2.5 的任务不是重新设计样本，而是把现有测试资产正式结构化沉淀下来。

验收标准

-

A1-E3 样本全部入库

-
-

字段统一

-
-

支持脚本直接读取

-
-

后续可扩展新增样本

•

4.2 T2：批量评测执行模块

职责

读取测试集，自动请求后端接口，采集结果并落表。

执行流程

读取测试集
→ 遍历样本
→ POST /analyze
→ 获取最终 JSON
→ 提取核心字段
→ 写入结果文件

输出结果建议字段

字段	含义
sample_id	样本编号
expected_stage	预期阶段
actual_stage	实际阶段
expected_interest	预期兴趣等级
actual_interest	实际兴趣等级
r1_correct	R1 是否严格正确
suggestion_aligned	建议是否对齐
next_step_conservative	next_step 是否克制
issue_type	问题类型
raw_output	原始返回结果

验收标准

•

15 条样本可一键批量跑完

•

-

无需人工逐条输入前端

-

-

自动产出表格文件

-

-

输出结构稳定

-

4.3 T3：指标统计与错误分类模块

职责

对批量测试结果进行自动统计与基础归类。

核心指标

-

Stage 准确率

-

-

R1 严格准确率

-

-

建议对齐率

-

-

next_step 克制率

-

当前基线

Day 7 当前结果为：

-

R1 严格准确率：46.7%

-

-

建议对齐率：93.3%

-

-

next_step 克制率：40.0%

-

-

全链路通过率：40.0%

-

这些数值应作为 V2.5 第一轮优化的对照基线。

错误分类建议

至少先支持以下类型：

-

边界失守

-

-

误判初识

-

-

interest_level 低估

-
-

interest_level 高估

-
-

建议漂移

-
-

next_step 不克制

-

验收标准

-

能自动计算核心指标

-
-

能自动初步打标签

-
-

能支持人工二次复核

-

4.4 R1：规则稳定化优化模块

职责

基于自动评测结果，围绕误差簇修复规则边界。

当前优先问题

根据 Day 7，总体优先级如下：

优先级	问题	说明
P1	E 类边界失守	E2/E3 被误吸入初识期
P2	interest_level 分档不稳	中间档位容易上下漂移
P3	next_step 不克制	产品表达层问题最突出

优化要求

R1 不再做“大改”，而是做边界精修：

-

强化“无法判断”触发条件

-

-

收紧“初识期兜底”的吸附范围

-

-

重定义初识/暧昧/拉扯不同阶段的兴趣分档规则

-

验收标准

-

E 类通过率显著提升

-

-

R1 严格准确率高于 Day 7 基线

-

-

规则可解释、可回归、可复盘

-

4.5 S5: 建议收敛优化模块

职责

继续强化 S5 对 R1 的服从，并压缩 next_step 的默认推进倾向。

当前问题

Day 7 最突出的产品层问题，不是建议完全失控，而是：

next_step 过于容易默认“再推进一步”。

优化方向

建立更强的阶段约束：

阶段	next_step 要求
无法判断	观察，不加码
初识期	轻互动，不升级
暧昧期 / 中	顺势承接，不提速
拉扯期	降压，不追问
冷淡期	止损，先退一步

验收标准

-
- next_step 克制率明显提升
-
-
- 不再普遍出现默认推进
-
-
- 建议文风自然，但边界更稳
-

4.6 P1：前端展示轻收口模块

职责

在规则与建议相对稳定后，适配更自然的产品展示。

说明

前端不再承担批量测试角色，只承担：

-

页面展示验证

-
-

少量代表样本体验测试

-
-

输出阅读友好度优化

-

验收标准

-

字段展示清晰

-
-

状态区分明确

-
-

自然语言可读

-
-

不影响后端与评测系统独立运行

-

五、V2.5 分阶段实施安排

第 1 天：测试体系设计与测试集结构化

目标

完成 V2.5 的评测基础设施设计。

当天任务

- 1.

整理 A/B/C/D/E 样本

- 2.

- 3.

固定测试集字段协议

- 4.

- 5.

建立测试集 JSON/CSV 文件

- 6.

- 7.

明确自动评测输出字段

- 8.

- 9.

冻结核心指标统计口径

10.

当日产出

-

测试集文件

-
-

指标定义文档

-
-

错误分类字典初版

-

验收标准

-

15 条样本全部结构化

-
-

预期结果统一

-
-

不再依赖聊天记录人工找样本

-

第 2 天：批量评测脚本实现

目标

实现自动跑样本、自动落表。

当天任务

1.

编写脚本读取测试集

2.

3.

自动请求 /analyze

4.

5.

解析结果

6.

7.

输出 CSV/JSON 结果表

8.

9.

补充异常兜底与失败重试

10.

当日产出

•

batch_test.py

•

•

自动结果表

•

•

运行日志样例

-

验收标准

-

一键跑完整批测试

-

-

不用手工输前端

-

-

输出表格完整可用

-

第 3 天：指标统计与错误分类落地

目标

让每轮测试都能自动出结论，而不是靠人工复盘。

当天任务

- 1.

计算 Stage 准确率

- 2.

- 3.

计算 R1 严格准确率

- 4.

5.

计算建议对齐率

6.

7.

计算 next_step 克制率

8.

9.

自动生成初步错误分类

10.

当日产出

-

metrics_report.json / csv

-

-

错误分类统计表

-

-

当前版本基线报告

-

验收标准

-

跑完样本后自动出统计结果

-

-

与 Day 7 手工结果口径一致

-

第 4 天：E 类边界专项修复

目标

优先修复最危险的边界失守问题。

当天任务

- 1.

梳理 E1/E2/E3 差异

- 2.

- 3.

收紧“无法判断”触发条件

- 4.

- 5.

限制初识期兜底吸附

- 6.

- 7.

跑专项回归

- 8.

当日产出

-

R1 规则修订版

-

-

E 类专项测试表

-

验收标准

-

E 类通过率显著提升

-

-

E2/E3 不再轻易误吸入初识期

-

第 5 天：interest_level 分档专项修复

目标

解决当前 R1 最主要的精度问题。

当天任务

- 1.

复盘 A2/A3/B2/B3/C3 错误

- 2.

- 3.

重写中间档规则边界

- 4.

- 5.

增加“正向承接强度”与“后退信号”联动约束

- 6.

7.

批量回归验证

8.

当日产出

-

interest_level 新规则表

-

-

修复前后对比表

-

验收标准

-

R1 严格准确率明显提升

-

-

中间档位误差收敛

-

第 6 天：next_step 收敛专项修复

目标

解决当前最突出的产品表达问题。

当天任务

1.

复盘所有 next_step 不克制样本

2.

3.

调整 S5 Prompt 约束

4.

5.

增加阶段到动作的硬边界

6.

7.

重新批量测试

8.

当日产出

-

S5 Prompt 收敛版

-

-

next_step 专项对比表

-

验收标准

-

next_step 克制率明显提升

-

-

默认推进倾向下降

•

第 7 天：V2.5 阶段验收与产品轻收口

目标

完成 V2.5 第一轮阶段性验收。

当天任务

1.

重跑全量样本

2.

3.

对比 Day 7 基线

4.

5.

输出阶段总结报告

6.

7.

抽取少量样本检查前端展示

8.

9.

明确是否进入下一轮优化或产品展示阶段

10.

当日产出

•

V2.5 验收测试报告

-
-

核心指标前后对比

-
-

下一阶段建议

-

验收标准

-

自动评测链完整

-
-

R1 严格准确率高于基线

-
-

next_step 克制率高于基线

-
-

手工测试工作量显著下降

-

六、验收指标与阶段目标

6.1 V2.5 验收维度

维度	基线	V2.5 目标
自动评测覆盖率	0%	100%
R1 严格准确率	46.7%	明显高于基线
建议对齐率	93.3%	保持或更高
next_step 克制率	40.0%	明显高于基线
手工测试占比	很高	大幅下降

基线来自 Day 7 全量测试结果。

6.2 V2.5 的核心验收标准

若满足以下条件，则可认为 V2.5 阶段成功：

1.

批量评测可一键运行

2.

3.

测试结果可自动生成表格

4.

5.

指标可自动统计

6.

7.

E 类边界明显收紧

8.

9.

interest_level 分档明显改善

10.

11.

next_step 明显收敛

12.

13.

前端不再承担批量测试职责

14.

七、风险与控制策略

7.1 风险一：自动化评测落地后，人工仍不放心

控制策略：前两轮允许自动表 + 人工复核并行，逐步建立信任。

7.2 风险二：规则优化过度，导致其他样本退化

控制策略：每次修改后必须全量回归，而非只看专项样本。

7.3 风险三：S5 收敛过头，导致建议过于僵硬

控制策略：保持“边界更稳”优先于“语言更灵活”，但保留少量自然表达空间。

7.4 风险四：过早回到前端打磨，分散主矛盾

控制策略：前端展示优化只能排在自动评测和规则修复之后。

八、阶段结论

LoveAdvisor 当前最需要的，不是再写一版更长的 prompt，也不是继续靠前端逐条手工测样本，而是完成一次开发方法的升级。

V2 已经完成了主链路搭建与规则化重构。V2.5 的意义，在于把这条链路纳入一个可量化、可回归、可持续优化的开发体系中。其重点不是扩功能，而是补齐“测试工程化”这一缺口，并围绕 Day 7 已暴露的误差簇进行精准修复。

最终目标不是让项目“看起来更复杂”，而是让它：

-

更稳

-
-

更准

-
-

更省力

-
-

更能持续迭代

-

九、最终阶段命名建议

建议将当前阶段正式命名为：

LoveAdvisor V2.5：评测驱动的稳定化开发阶段

这个命名比继续沿用“Day 8 / Day 9”更准确，因为它反映的不是节点开发顺序，而是项目开发方法论的升级。

如果你要，我下一条可以直接继续给你两样最落地的东西之一：

《V2.5 第一天实施清单》，或者 **《自动评测系统的数据结构 + 脚本设计方案》**。

下面给你一版正式可执行、适合项目归档的《LoveAdvisor V2.5 正式实施方案》。

这版不是延续之前那种“按节点继续补”，而是基于你们已经暴露出来的真实问题，转向评测驱动的稳定化开发。其核心目的，是解决当前开发低效、测试手工化、规则优化缺乏量化依据的问题。方案是在 V2 已完成架构重整、信号层拆分、R1 规则化、S5 重接入、Day 7 全量评估基础上提出的。

LoveAdvisor V2.5 正式实施方案
评测驱动的稳定化开发阶段

一、项目阶段重新定义

1.1 当前阶段判断

LoveAdvisor 项目截至 V2 Day 7，已经完成了以下关键工作：

完成 V1 的链路打通验证

完成 V2 的架构重整与职责重构

建立 S2 基础互动信号层

建立 S3 高级关系信号层

实现 R1 后端规则引擎

完成 S5 建议层与规则层对齐

打通 O1 输出层与前后端展示链路

初步形成全量测试与错误分类体系

因此，项目已经不再处于“从 0 到 1 搭系统”的阶段，而是进入了一个新的阶段：系统已可运行，但尚未形成高效、科学、可持续的优化机制。当前的主要瓶颈不再是“链路通不通”，而是：

批量测试方式低效，严重依赖手工

规则优化缺少自动化评测支撑

E 类边界、interest_level、next_step 仍存在系统性误差

前端承担了不该承担的测试工作

迭代方式仍偏经验驱动，而非指标驱动

基于此，项目正式进入：

LoveAdvisor V2.5：评测驱动的稳定化开发阶段

1.2 V2.5 的核心目标

V2.5 的目标不是继续扩功能，也不是继续堆节点，而是解决一个更根本的问题：让项目从“能迭代”升级为“能高效、可量化、可持续地迭代”。

具体包括四个目标：

目标编号 目标名称 核心内容

- G1 测试工程化 将 Day 7 式手工评测改为自动评测
- G2 规则稳定化 围绕核心误差簇修复规则边界
- G3 建议收敛化 压缩建议层尤其是 next_step 的默认推进倾向
- G4 产品轻收口 在稳定化之后再优化展示与用户体验

1.3 V2.5 与 V1 / V2 的区别

阶段 目标 主导方式 核心产物

- V1 打通链路、验证可行性 工作流驱动 可运行 Demo
- V2 重构职责、规则增强 架构驱动 稳定主链路
- V2.5 量化优化、提升迭代效率 评测驱动 自动评测体系 + 稳定化版本

一句话概括：

V1 解决“能不能做”，V2 解决“怎么做稳”，V2.5 解决“怎么高效地把它做好”。

二、V2.5 的总体开发原则

2.1 原则一：开发与评测彻底分离

当前开发中最明显的低效点，是把“前端手工输入样本”当成评测方法。这会导致：

批量测试效率极低

结果整理重复劳动严重

每轮改动无法快速回归

误差分析高度依赖人工体力

因此，V2.5 的第一原则是：

前端负责展示，脚本负责评测。

以后开发流程必须拆为三层：

开发层：改 prompt、改规则、改后端逻辑

评测层：自动跑样本、自动生成表格、自动分类错误

展示层：仅查看少量代表样本的产品表现

2.2 原则二：优化必须由指标驱动

Day 7 已经形成了比较清晰的判定维度：

R1 是否正确

建议是否对齐

next_step 是否克制

问题类型分类

V2.5 要把它们从“人工复盘口径”升级为“正式开发指标”。

建议固定以下四项核心指标：

指标	含义	作用
Stage 准确率	relationship_stage 是否正确	判断主标签稳定性
R1 严格准确率	stage + interest_level 同时正确	判断规则层整体质量
建议对齐率	S5 是否服从 R1 结果	判断建议层是否越权
next_step 克制率	最终动作是否符合阶段边界	判断产品表达是否成熟

所有后续优化，必须明确对应至少一个指标提升目标。

2.3 原则三：按误差簇优化，而不是按节点机械推进

V2 阶段是按节点推进的，这在架构建设期是合理的。

但到了 V2.5，继续机械地“今天改 S2、明天改 S3”会越来越低效。

因为 Day 7 已经证明，当前问题已经收敛到几个明确误差簇：

E 类边界失守

interest_level 中间档不稳

next_step 普遍不够克制

因此，V2.5 的优化顺序必须是：
先看误差集中在哪，再决定修哪一层。

2.4 原则四：不新增复杂链路，优先固化现有链路

当前主链路已经基本形成：

前端输入 → 后端调用 → Coze 工作流 → S2/S3 → R1 → S5 → O1 → 前端展示。

V2.5 阶段不再优先新增新节点、新模型分工、新产品模块。

所有工作优先围绕以下三件事展开：

测试是否能自动跑

规则是否能稳定修

表达是否能明显收敛

三、V2.5 总体架构设计

3.1 业务主链路保持不变

V2.5 不重构业务主链路，继续沿用 V2 已验证的结构：

用户输入

→ S1 文本清洗

→ S2 基础互动信号提取

→ S3 高级关系信号提取

→ R1 后端规则引擎

→ S5 策略建议生成

→ O1 输出组装层

→ 前端展示

原因是：

当前主问题并不在“链路设计错误”，而在“链路优化与评测方式低效”。

3.2 新增“评测子系统”

V2.5 最大的新增部分，不是业务链路，而是旁路的评测系统。

新增结构如下：

测试集文件

→ 批量评测脚本

→ 自动请求 /analyze

→ 自动提取返回结果

→ 自动生成测试结果表

→ 自动统计核心指标

→ 自动输出错误分类
这套子系统不影响产品主链路，但直接决定开发效率。

3.3 评测系统的定位
评测系统的职责包括：

管理标准测试样本

固定预期结果

一键跑全量样本

自动生成结果记录

自动判定主要指标

为规则修改提供客观对比依据

它的作用不是“替代人工判断”，而是：
把机械重复劳动从人身上剥离出来，让人只做真正需要判断的事情。

四、V2.5 核心模块设计

4.1 T1：测试集管理模块

职责

统一管理所有回归测试样本及其预期结果。

输入结构建议

每条样本至少包含：

```
{
  "sample_id": "A1",
  "group": "A",
  "input_text": "A： 你好\nB： 你好\nA： 你是哪个专业的\nB： 金融",
  "expected_relationship_stage": "初识期",
  "expected_interest_level": "低",
  "expected_suggestion_aligned": true,
  "expected_next_step_style": "克制"
}
```

说明

V2 阶段已有 A/B/C/D/E 五组测试资产，Day 7 也已经形成了 15 条样本的完整测试表。V2.5

的任务不是重新设计样本，而是把现有测试资产正式结构化沉淀下来。
验收标准

A1-E3 样本全部入库

字段统一

支持脚本直接读取

后续可扩展新增样本

4.2 T2：批量评测执行模块

职责

读取测试集，自动请求后端接口，采集结果并落表。

执行流程

读取测试集

→ 遍历样本

→ POST /analyze

→ 获取最终 JSON

→ 提取核心字段

→ 写入结果文件

输出结果建议字段

字段	含义
sample_id	样本编号
expected_stage	预期阶段
actual_stage	实际阶段
expected_interest	预期兴趣等级
actual_interest	实际兴趣等级
r1_correct	R1 是否严格正确
suggestion_aligned	建议是否对齐
next_step_conservative	next_step 是否克制
issue_type	问题类型
raw_output	原始返回结果

验收标准

15 条样本可一键批量跑完

无需人工逐条输入前端

自动产出表格文件

输出结构稳定

4.3 T3：指标统计与错误分类模块

职责

对批量测试结果进行自动统计与基础归类。

核心指标

Stage 准确率

R1 严格准确率

建议对齐率

next_step 克制率

当前基线

Day 7 当前结果为：

R1 严格准确率：46.7%

建议对齐率：93.3%

next_step 克制率：40.0%

全链路通过率：40.0%

这些数值应作为 V2.5 第一轮优化的对照基线。

错误分类建议

至少先支持以下类型：

边界失守

误判初识

interest_level 低估

interest_level 高估

建议漂移

next_step 不克制

验收标准

能自动计算核心指标

能自动初步打标签

能支持人工二次复核

4.4 R1：规则稳定化优化模块

职责

基于自动评测结果，围绕误差簇修复规则边界。

当前优先问题

根据 Day 7，总体优先级如下：

优先级	问题	说明
P1	E 类边界失守	E2/E3 被误吸入初识期
P2	interest_level 分档不稳	中间档位容易上下漂移
P3	next_step 不克制	产品表达层问题最突出

优化要求

R1 不再做“大改”，而是做边界精修：

强化“无法判断”触发条件

收紧“初识期兜底”的吸附范围

重定义初识/暧昧/拉扯不同阶段的兴趣分档规则

验收标准

E 类通过率显著提升

R1 严格准确率高于 Day 7 基线

规则可解释、可回归、可复盘

4.5 S5：建议收敛优化模块

职责

继续强化 S5 对 R1 的服从，并压缩 next_step 的默认推进倾向。

当前问题

Day 7 最突出的产品层问题，不是建议完全失控，而是：

next_step 过于容易默认“再推进一步”。

优化方向

建立更强的阶段约束：

阶段 next_step 要求

无法判断 观察，不加码

初识期 轻互动，不升级

暧昧期 / 中 顺势承接，不提速

拉扯期 降压，不追问

冷淡期 止损，先退一步

验收标准

next_step 克制率明显提升

不再普遍出现默认推进

建议文风自然，但边界更稳

4.6 P1：前端展示轻收口模块

职责

在规则与建议相对稳定后，适配更自然的产品展示。

说明
前端不再承担批量测试角色，只承担：

页面展示验证

少量代表样本体验测试

输出阅读友好度优化

验收标准

字段展示清晰

状态区分明确

自然语言可读

不影响后端与评测系统独立运行

五、V2.5 分阶段实施安排

第 1 天：测试体系设计与测试集结构化
目标
完成 V2.5 的评测基础设施设计。
当天任务

整理 A/B/C/D/E 样本

固定测试集字段协议

建立测试集 JSON/CSV 文件

明确自动评测输出字段

冻结核心指标统计口径

当日报出

测试集文件

指标定义文档

错误分类字典初版

验收标准

15 条样本全部结构化

预期结果统一

不再依赖聊天记录人工找样本

第 2 天：批量评测脚本实现
目标
实现自动跑样本、自动落表。
当天任务

编写脚本读取测试集

自动请求 `/analyze`

解析结果

输出 CSV/JSON 结果表

补充异常兜底与失败重试

当日报出

batch_test.py

自动结果表

运行日志样例

验收标准

一键跑完整批测试

不用手工输前端

输出表格完整可用

第 3 天：指标统计与错误分类落地

目标

让每轮测试都能自动出结论，而不是靠人工复盘。

当天任务

计算 Stage 准确率

计算 R1 严格准确率

计算建议对齐率

计算 next_step 克制率

自动生成初步错误分类

当日报出

metrics_report.json / csv

错误分类统计表

当前版本基线报告

验收标准

跑完样本后自动出统计结果

与 Day 7 手工结果口径一致

第 4 天：E 类边界专项修复

目标

优先修复最危险的边界失守问题。

当天任务

梳理 E1/E2/E3 差异

收紧“无法判断”触发条件

限制初识期兜底吸附

跑专项回归

当日报出

R1 规则修订版

E 类专项测试表

验收标准

E 类通过率显著提升

E2/E3 不再轻易误吸入初识期

第 5 天：interest_level 分档专项修复

目标

解决当前 R1 最主要的精度问题。

当天任务

复盘 A2/A3/B2/B3/C3 错误

重写中间档规则边界

增加“正向承接强度”与“后退信号”联动约束

批量回归验证

当日报出

interest_level 新规则表

修复前后对比表

验收标准

R1 严格准确率明显提升

中间档位误差收敛

第 6 天: next_step 收敛专项修复

目标

解决当前最突出的产品表达问题。

当天任务

复盘所有 next_step 不克制样本

调整 S5 Prompt 约束

增加阶段到动作的硬边界

重新批量测试

当日产出

S5 Prompt 收敛版

next_step 专项对比表

验收标准

next_step 克制率明显提升

默认推进倾向下降

第 7 天：V2.5 阶段验收与产品轻收口

目标

完成 V2.5 第一轮阶段性验收。

当天任务

重跑全量样本

对比 Day 7 基线

输出阶段总结报告

抽取少量样本检查前端展示

明确是否进入下一轮优化或产品展示阶段

当日产出

V2.5 验收测试报告

核心指标前后对比

下一阶段建议

验收标准

自动评测链完整

R1 严格准确率高于基线

next_step 克制率高于基线

手工测试工作量显著下降

六、验收指标与阶段目标

6.1 V2.5 验收维度

维度	基线	V2.5 目标
----	----	---------

自动评测覆盖率	0%	100%
---------	----	------

R1 严格准确率	46.7%	明显高于基线
----------	-------	--------

建议对齐率	93.3%	保持或更高
-------	-------	-------

next_step 克制率	40.0%	明显高于基线
---------------	-------	--------

手工测试占比	很高	大幅下降
--------	----	------

基线来自 Day 7 全量测试结果。

6.2 V2.5 的核心验收标准

若满足以下条件，则可认为 V2.5 阶段成功：

批量评测可一键运行

测试结果可自动生成表格

指标可自动统计

E 类边界明显收紧

interest_level 分档明显改善

next_step 明显收敛

前端不再承担批量测试职责

七、风险与控制策略

7.1 风险一：自动化评测落地后，人工仍不放心

控制策略：前两轮允许自动表 + 人工复核并行，逐步建立信任。

7.2 风险二：规则优化过度，导致其他样本退化

控制策略：每次修改后必须全量回归，而非只看专项样本。

7.3 风险三：S5 收敛过头，导致建议过于僵硬

控制策略：保持“边界更稳”优先于“语言更灵活”，但保留少量自然表达空间。

7.4 风险四：过早回到前端打磨，分散主矛盾

控制策略：前端展示优化只能排在自动评测和规则修复之后。

八、阶段结论

LoveAdvisor 当前最需要的，不是再写一版更长的 prompt，也不是继续靠前端逐条手工测样本，而是完成一次开发方法的升级。

V2 已经完成了主链路搭建与规则化重构。V2.5 的意义，在于把这条链路纳入一个可量化、可回归、可持续优化的开发体系中。其重点不是扩功能，而是补齐“测试工程化”这一缺口，并围绕 Day 7 已暴露的误差簇进行精准修复。

最终目标不是让项目“看起来更复杂”，而是让它：

更稳

更准

更省力

更能持续迭代

九、最终阶段命名建议

建议将当前阶段正式命名为：

LoveAdvisor V2.5：评测驱动的稳定化开发阶段

这个命名比继续沿用“Day 8 / Day 9”更准确，因为它反映的不是节点开发顺序，而是项目开发方法论的升级。

如果你要，我下一条可以直接继续给你两样最落地的东西之一：

《V2.5 第一天实施清单》，或者 《自动评测系统的数据结构 + 脚本设计方案》。